

DETAILED PROJECT REPORT

SLAM Research & Development for GPS-Denied UAV Navigation

Evaluation of Visual SLAM Frameworks and a Proposed Integration Path into the Drone Platform and AERO-GCS

Prepared by: Aadarsh
Autonomous Systems / Drone Software Engineering
July 2026

Document Classification: Internal — Engineering R&D

Table of Contents

1. Executive Summary	3
2. Introduction & Background	3
3. Objectives	3
4. Understanding SLAM: Core Concepts	4
4.1 The SLAM Problem	4
4.2 Front-End vs Back-End Architecture	4
4.3 Categories of SLAM Approaches	4
4.4 The Pipelines Studied: ORB-SLAM3 and RTAB-Map	5
5. Glossary of Terms	5
6. Research Methodology & Activities	8
6.1 Literature Review	8
6.2 Framework Evaluation: ORB-SLAM3 vs RTAB-Map	8
6.3 Development Environment	8
7. System Architecture Studied	8
8. Camera & Sensor Study	8
8.1 Camera Types and Acquisition Methods	8
8.2 Calibration, Distortion, and Synchronization	9
9. Knowledge Gained	9
10. Proposed Integration: SLAM on the Drone Platform	9
10.1 Hardware Architecture	10
10.2 Software Stack & Data Flow	10
10.3 Integration with ArduPilot / MAVLink	10
10.4 Integration with AERO-GCS	11
10.5 Phased Implementation Plan	11
10.6 Use Case: SAR / GPS-Denied Operations	11
10.7 Safety & Failover Considerations	12
11. Risks & Challenges	12
12. Future Scope	12
13. Conclusion	13
14. References & Further Reading	13

1. Executive Summary

This report documents research undertaken into Simultaneous Localization and Mapping (SLAM) as a candidate technology for GPS-denied autonomous navigation on our UAV platforms. It expands on the original research summary in three ways: (a) it explains SLAM concepts and terminology in depth so the work is legible to engineers outside the immediate research effort, (b) it analyses the frameworks evaluated — ORB-SLAM3 and RTAB-Map — and the development environment used to test them, and (c) it proposes a concrete, phased path to integrating SLAM-based localization into the drone hardware, ArduPilot's navigation stack, and AERO-GCS.

The research phase did not produce a flight-ready system; it produced something arguably more valuable at this stage — a validated understanding of the problem, a working development environment, and a clear-eyed view of what stands between the current state and an onboard implementation. This document is written to carry that understanding forward into an implementation project.

2. Introduction & Background

Almost all of our current autonomous flight capability — AERO-GCS mission planning, the SAR detection pipeline — assumes a usable GPS fix. That assumption breaks down in exactly the environments where autonomy is most valuable to a rescue or defense-adjacent operator: inside collapsed structures, under dense forest canopy, in tunnels and parking structures, in urban canyons between tall buildings, and in any GPS-denied or GPS-degraded scenario including intentional jamming or spoofing.

SLAM is the standard robotics answer to this problem. Rather than relying on an external positioning signal, the vehicle uses onboard sensors — typically a camera and/or an inertial measurement unit — to simultaneously (a) estimate its own trajectory and (b) build a map of the space it is moving through. The 'simultaneous' part is what makes the problem hard: an accurate map requires knowing where the sensor was when each observation was made, and an accurate position estimate requires already having a map to localize against. SLAM resolves this circularity through incremental estimation and periodic global correction, explained in detail in Section 4.

This work focused specifically on studying open-source SLAM frameworks and preparing a practical implementation pipeline for future UAV applications, rather than jumping directly to an onboard flight test — a sequencing choice that is standard practice in robotics R&D, where the cost of debugging a SLAM stack in the air is far higher than debugging it on a bench.

3. Objectives

- Build a working understanding of SLAM fundamentals: the localization/mapping problem, front-end/back-end architecture, and the tradeoffs between filter-based and graph-based formulations.
- Evaluate two candidate open-source SLAM frameworks — ORB-SLAM3 and RTAB-Map — against criteria relevant to our platform: robustness, ROS integration, sensor flexibility, and community/maintenance status.
- Stand up a working Linux robotics development environment (Ubuntu under WSL) capable of building and running modern SLAM stacks.

- Gain working knowledge of ROS 2 (Jazzy) as the middleware layer connecting sensors, SLAM, and downstream consumers.
- Understand camera calibration and sensor characteristics well enough to select and configure a camera for future onboard use.
- Scope how SLAM could be integrated into autonomous drone navigation and into our custom Ground Control Station (AERO-GCS).

4. Understanding SLAM: Core Concepts

This section explains how SLAM works in general, independent of any specific framework, so the rest of the report can be read without external references.

4.1 The SLAM Problem

A vehicle moving through an unknown environment observes the world through its sensors (here, a camera). From those observations it must recover two things at once: where it is (localization) and what the environment looks like (mapping). Neither can be solved cleanly first — the map is built from poses, and the poses are estimated relative to the map — so SLAM algorithms solve both jointly, refining each estimate as new data arrives.

In practice this is handled by (1) making a fast, short-term estimate of motion between consecutive sensor frames, then (2) periodically correcting that estimate against a longer-term, globally consistent map once enough evidence has accumulated. This two-speed structure is the front-end/back-end split described below.

4.2 Front-End vs Back-End Architecture

The front-end runs at or near sensor frame rate. For visual SLAM it typically performs: feature extraction (finding distinctive, trackable points in each image), feature matching (associating those points with points seen in previous frames), and pose estimation (computing how the camera must have moved to explain the observed motion of those points — this frame-to-frame estimate is visual odometry). Because it only ever compares nearby frames, the front-end's estimate drifts steadily over time — small per-frame errors accumulate into large positional error over a long trajectory.

The back-end corrects that drift. It maintains a map of keyframes (a representative subset of frames) and 3D landmarks, and periodically runs an optimization — bundle adjustment over the local map, or pose graph optimization over the full trajectory — that adjusts past poses to be mutually consistent with everything observed so far. The single most powerful correction the back-end can apply is a loop closure: recognizing that the vehicle has returned to a place it has seen before, which lets the optimizer tie the current (drifted) pose estimate directly back to the earlier, more certain one and redistribute the accumulated error across the whole path in between.

4.3 Categories of SLAM Approaches

- Feature-based SLAM tracks a sparse set of distinctive image points (corners, blobs) frame to frame — computationally efficient and the basis of ORB-SLAM3.
- Direct / appearance-based methods work more broadly on image intensity or whole-image appearance rather than a small set of extracted features — RTAB-Map's loop-closure detector is appearance-based, comparing whole-image signatures to recognize revisited places.
- Graph-based SLAM represents the problem as a graph — poses and landmarks as nodes, sensor measurements as edges — and solves for the node configuration that best satisfies all edges

simultaneously. This is the dominant modern back-end formulation and the one both ORB-SLAM3 and RTAB-Map ultimately rely on.

- Filter-based SLAM (EKF-SLAM) maintains a single running estimate updated recursively with each new measurement. It was the original SLAM formulation but scales poorly with map size and has largely been superseded by graph-based optimization for anything beyond small maps.

4.4 The Pipelines Studied: ORB-SLAM3 and RTAB-Map

ORB-SLAM3 is a feature-based visual (and visual-inertial) SLAM system. Its pipeline — tracking, local mapping, loop closing — matches the general architecture above closely: ORB features are extracted per frame, matched against the local map to estimate pose, new keyframes trigger local bundle adjustment, and a separate loop-closing thread continuously checks for revisits and triggers global pose-graph correction when one is found. It supports monocular, stereo, and RGB-D input, and its "Atlas" system can maintain multiple disconnected sub-maps and merge them later — useful if tracking is briefly lost and later recovered.

RTAB-Map (Real-Time Appearance-Based Mapping) takes an appearance-based approach to its defining feature — loop closure detection — while still building a graph-based map underneath. It is designed from the ground up for real-time operation with a bounded memory footprint (older, less relevant parts of the map are moved to working memory rather than kept fully active), and it has substantially stronger native ROS/ROS 2 integration, multi-session mapping, and out-of-the-box support for RGB-D, stereo, and lidar input. This is consistent with the report's finding: RTAB-Map was selected over ORB-SLAM3 specifically for its ROS integration and suitability for future robotic (not just visual-research) applications — a reasonable call, since the end goal is a working ROS 2 node on a companion computer talking to ArduPilot, not a standalone visual-SLAM demo.

5. Glossary of Terms

The terminology below covers every technical term used in this report and the surrounding SLAM/robotics domain, for reference by anyone reviewing this document without a robotics background.

Term	Definition
SLAM	Simultaneous Localization and Mapping — the process of building a map of an unknown environment while simultaneously tracking a vehicle's position within that map, using only onboard sensors.
GPS-denied navigation	Navigation in environments where satellite positioning is unavailable or unreliable — indoors, under dense canopy, in tunnels, near tall structures, or during GNSS jamming/spoofing.
Localization	Estimating the vehicle's own position and orientation (pose) relative to a map or reference frame.
Mapping	Building a representation of the environment — as a sparse set of landmarks, a dense point cloud, or an occupancy grid — from sensor observations.
Front-end (SLAM)	The part of a SLAM pipeline that processes raw sensor data frame-by-frame: feature detection, matching, and short-term pose estimation (visual odometry).
Back-end (SLAM)	The part of a SLAM pipeline that refines the front-end's estimates over time using optimization — correcting drift via bundle adjustment or pose-graph optimization once loop closures are detected.

Visual SLAM (VSLAM)	SLAM performed primarily using camera imagery (monocular, stereo, or RGB-D) rather than lidar or radar.
Feature-based SLAM	A SLAM approach that tracks distinct, repeatable visual features (corners, blobs) across frames, rather than working on raw pixel intensities.
Graph-based SLAM	A SLAM formulation that represents robot poses and landmarks as nodes in a graph, with sensor measurements as edges (constraints); solved by finding the pose configuration that best satisfies all constraints.
Filter-based SLAM (EKF-SLAM)	An older SLAM formulation using an Extended Kalman Filter to recursively update a single best estimate of pose and map; largely superseded by graph-based/optimization methods for larger maps.
ORB-SLAM3	An open-source, feature-based visual (and visual-inertial) SLAM system supporting monocular, stereo, and RGB-D cameras, known for robust tracking, loop closing, and multi-map ("Atlas") capability.
ORB features	"Oriented FAST and Rotated BRIEF" — a fast, rotation-invariant corner-detection and description algorithm used by ORB-SLAM to identify and match trackable points between frames.
Feature extraction	Detecting distinctive, repeatable points (corners, edges) in an image that can be reliably re-identified in subsequent frames.
Feature matching / tracking	Associating features detected in the current frame with features seen in previous frames or in the map, to estimate how the camera has moved.
Visual odometry (VO)	Estimating incremental camera motion (frame-to-frame) from image data alone. VO alone accumulates drift over time; SLAM adds mapping and loop closure to bound that drift.
Local mapping	The process of triangulating newly observed features into 3D map points and refining nearby keyframes and map points as new data arrives.
Loop closure	Recognizing that the vehicle has returned to a previously visited location, allowing the back-end to correct accumulated drift by tying the current pose back to the earlier one.
Bundle adjustment	A nonlinear optimization that jointly refines camera poses and 3D map point positions to minimize the total reprojection error across many observations.
Keyframe	A representative frame selected from the video stream (not every frame) to anchor the map and reduce computational load, chosen when the camera has moved or observed enough new information.
Pose graph optimization	Back-end optimization that adjusts a graph of keyframe poses to be globally consistent with all measured relative-motion and loop-closure constraints.
Occupancy grid	A map representation that divides space into cells marked free, occupied, or unknown — commonly used for path planning and obstacle avoidance.
Point cloud	A dense set of 3D points representing surfaces in the environment, typically generated from stereo or RGB-D depth data.
RTAB-Map	"Real-Time Appearance-Based Mapping" — an open-source graph-based SLAM library with strong loop-closure detection, multi-session mapping, and native ROS/ROS 2 integration; supports RGB-D, stereo, and lidar input.
ROS 2 (Jazzy)	Robot Operating System 2 — a middleware framework providing communication, drivers, and tooling for robotics software; "Jazzy Jalisco" is a

	specific long-term-support release used in this work.
Middleware / DDS	The publish-subscribe communication layer (Data Distribution Service) underlying ROS 2, which lets independent nodes (camera driver, SLAM node, flight controller bridge) exchange data in real time.
WSL (Windows Subsystem for Linux)	A compatibility layer allowing a Linux environment (e.g., Ubuntu) to run under Windows — useful for early development, but with known limitations for USB device passthrough (e.g., cameras) relevant to onboard robotics work.
Camera calibration	The process of determining a camera's intrinsic parameters and lens-distortion coefficients so pixel measurements can be converted into accurate real-world geometry.
Intrinsic parameters	Camera-specific values (focal length, principal point, skew) describing how 3D points project onto the 2D image sensor, independent of camera position.
Extrinsic parameters	The position and orientation of the camera (or of one sensor relative to another, e.g., camera-to-IMU) in a common reference frame.
Lens distortion	Deviation of a real lens from an ideal pinhole model (radial and tangential distortion), which must be corrected for accurate feature geometry.
Monocular / Stereo / RGB-D camera	Monocular: single camera, no direct depth (scale must be inferred or fused with IMU). Stereo: two cameras compute depth via disparity. RGB-D: a colour camera paired with an active depth sensor (structured light or ToF).
IMU (Inertial Measurement Unit)	A sensor combining accelerometers and gyroscopes to measure linear acceleration and angular rate, used to aid SLAM through fast-rate motion estimation between camera frames (visual-inertial odometry).
Sensor fusion	Combining measurements from multiple sensors (camera, IMU, GPS, barometer) to produce a single, more accurate and robust state estimate than any sensor alone.
EKF (Extended Kalman Filter)	A recursive estimator used by autopilots (e.g., ArduPilot's EKF3) to fuse IMU, GPS, barometer, and optionally external vision/SLAM data into a single navigation solution.
MAVLink	The lightweight messaging protocol used between ground control stations, companion computers, and flight controllers such as ArduPilot and PX4.
MAVROS	A ROS/ROS 2 package that bridges MAVLink messages into the ROS ecosystem, allowing a companion computer running SLAM to exchange data with the flight controller.
VISION_POSITION_ESTIMATE / ODOMETRY	MAVLink messages used to feed externally computed pose (e.g., from SLAM) into ArduPilot's EKF as a substitute or supplement for GPS.
ArduPilot	The open-source autopilot firmware already used in AERO-GCS's target platforms, supporting external navigation sources via its EKF3 source-selection parameters.
Drift	Accumulated position/orientation error that grows over time or distance in dead-reckoning-style estimation (e.g., raw visual or inertial odometry) in the absence of correcting measurements like loop closure or GPS.

6. Research Methodology & Activities

6.1 Literature Review

A literature review covered the three broad families of SLAM technique — feature-based, direct/visual, and graph-based — to establish the vocabulary and design tradeoffs before touching any code. This grounding is what makes it possible to read either framework's source or documentation and understand what each module is doing and why.

6.2 Framework Evaluation: ORB-SLAM3 vs RTAB-Map

ORB-SLAM3 was built and explored first, including its feature extraction, tracking, local mapping, and loop closure modules. Getting it running surfaced a realistic set of robotics-software friction: dependency and build issues across OpenCV (computer vision primitives), Eigen (linear algebra used throughout the optimizer), and Pangolin (the 3D visualization library ORB-SLAM3 uses for its map viewer). These are exactly the dependencies most open-source robotics research code shares, so resolving them here is directly reusable experience for any future C++/vision work on this platform.

RTAB-Map was evaluated next and selected as the primary path forward, on the strength of its ROS 2 integration and its better fit for a robotic (as opposed to purely research/visual) end goal — namely, feeding a live pose estimate to a flight controller rather than producing an offline trajectory for analysis.

6.3 Development Environment

Ubuntu, ROS 2 Jazzy, and RTAB-Map were configured under WSL (Windows Subsystem for Linux), resolving a series of dependency and build issues along the way. This is a reasonable environment for early framework evaluation and ROS 2 familiarization, but it is worth flagging explicitly for the next phase: WSL has known limitations around USB device passthrough, which matters as soon as a physical camera needs to be tested rather than played back from a recorded dataset. Section 10 recommends moving to native Ubuntu on the actual onboard companion computer before hardware-in-the-loop testing begins, rather than trying to extend the WSL environment to cover that case.

7. System Architecture Studied

The architecture studied traces the full SLAM data path: camera input, feature extraction, feature matching, pose estimation, keyframe generation, map (local) optimization, loop closure detection, pose graph (global) optimization, and map visualization. Understanding how sensors, the SLAM algorithm, and the ROS 2 middleware interact — which node publishes what, at what rate, and who consumes it — is what makes it possible to reason about where a flight controller bridge and a GCS visualization layer would plug into this pipeline, which is the subject of Section 10.

8. Camera & Sensor Study

8.1 Camera Types and Acquisition Methods

Different methods of obtaining camera data were evaluated, including USB-connected cameras and mobile-phone IP camera solutions (streaming over the network). For any onboard, real-time use case, a directly-connected USB or MIPI camera is strongly preferable to a network-streamed source: IP camera links introduce variable latency and compression artifacts, both of which directly degrade

feature tracking and pose estimation quality. Network cameras remain useful for quick desk-based experimentation but should not be treated as representative of onboard performance.

Looking ahead to Section 10, the camera type chosen materially changes what the SLAM system can do: a monocular camera alone cannot recover absolute scale (it knows shape, not size, unless fused with an IMU or another metric sensor); a stereo pair recovers metric depth directly from the known baseline between the two lenses; an RGB-D camera (colour plus an active depth sensor) gives dense depth per pixel but typically has a shorter effective range and can struggle outdoors in bright sunlight. This tradeoff is discussed further in Section 10.1.

8.2 Calibration, Distortion, and Synchronization

Camera calibration, intrinsic parameters, lens distortion, synchronization, and image quality were studied because they directly affect localization and mapping accuracy. Concretely: an uncalibrated or poorly-calibrated camera introduces systematic geometric error into every feature observation, which the SLAM optimizer will faithfully bake into the map and trajectory rather than correct — calibration errors do not average out, they compound. Synchronization matters most once an IMU is added: visual-inertial fusion assumes the camera frame and the IMU sample it is paired with were captured at (or very close to) the same instant, and a timestamp offset that seems trivial on the bench can meaningfully hurt fusion accuracy in the air.

9. Knowledge Gained

The work produced practical, hands-on understanding — not just conceptual familiarity — of the following:

- ORB features and how feature-based tracking degrades on low-texture or repetitive surfaces (a real concern for indoor SAR environments — bare walls, rubble).
- Visual odometry and why it drifts without correction.
- Bundle adjustment and keyframes as the mechanism for keeping local maps accurate without processing every single frame.
- Loop closure as the primary defence against long-run drift, and why environments with few revisit opportunities (e.g., a single sweep through a collapsed structure) are the hardest case for SLAM.
- Occupancy grids and pose graph optimization as the map representations most directly useful for path planning and obstacle avoidance downstream of SLAM itself.
- ROS-based robotics workflows — nodes, topics, launch files — as the integration layer that will carry SLAM output to the flight controller bridge.
- The practical importance of calibration quality, computational headroom, and sensor quality — these are not academic concerns, they are the difference between a SLAM system that works and one that silently drifts.

10. Proposed Integration: SLAM on the Drone Platform

This section moves beyond the original report's scope to lay out a concrete path from the current research foundation to an onboard, flight-tested capability, integrated with our existing ArduPilot-based airframes and AERO-GCS.

10.1 Hardware Architecture

SLAM at useful frame rates needs dedicated onboard compute — it cannot run on the flight controller itself. The recommended architecture is a companion computer, connected to the flight controller over a serial or USB link running MAVLink, mirroring the ground-side architecture already used for the SAR detection pipeline (RTSP inference on a separate compute node).

- Companion computer: an NVIDIA Jetson-class board (e.g., Orin Nano / Orin NX) is the natural choice — it has the GPU headroom to run RTAB-Map and, notably, could share compute with the existing YOLOv8 SAR detection model if both are ever needed on the same mission, since both are already CUDA-accelerated workloads in our stack.
- Camera: a stereo or RGB-D module with a built-in IMU (e.g., an Intel RealSense-class depth camera) is preferable to a plain monocular USB webcam — it removes the scale-ambiguity problem discussed in Section 8.1 and gives RTAB-Map a metric depth source directly, rather than relying solely on visual-inertial fusion for scale.
- IMU: the flight controller's own IMU can supplement or cross-check the camera-integrated IMU, but for tight time synchronization the camera's own IMU is generally the one fed directly into the SLAM node.
- Weight/power budget: this is an additive payload on top of the existing SAR/GCS hardware envelope (Skydroid H12 link, RTSP camera, etc.) and needs to be budgeted explicitly against current AUW and flight-time figures rather than assumed to be free.

10.2 Software Stack & Data Flow

The proposed software chain runs entirely on the companion computer, with only a thin telemetry bridge crossing to the flight controller:

- Camera + IMU driver publishes synchronized image and inertial data as ROS 2 topics.
- An `rtabmap_ros` node consumes those topics and publishes real-time odometry (`nav_msgs/Odometry`) plus, as compute allows, an occupancy grid and/or point cloud.
- A MAVROS bridge node converts that ROS 2 odometry into the corresponding MAVLink message (`VISION_POSITION_ESTIMATE` or `ODOMETRY`) and sends it to the flight controller over the existing serial/USB telemetry link.
- ArduPilot's EKF3 is configured (via its `EK3_SRC` source-selection parameters) to accept this external vision source as a position/velocity input, either replacing GPS entirely in fully GPS-denied flight or running alongside it for cross-checking.

10.3 Integration with ArduPilot / MAVLink

This is the step that turns 'SLAM running on a bench' into 'SLAM the autopilot actually trusts.' ArduPilot's EKF is already designed to fuse an external navigation source through the `VISION_POSITION_ESTIMATE` / `ODOMETRY` message pair — this is a well-trodden path for GPS-denied multirotor work, not a custom integration from scratch. The main engineering work on our side is (a) getting the MAVROS bridge configured and timestamp-aligned correctly, (b) tuning `EK3_SRC` and the associated noise parameters so the EKF weights vision input appropriately relative to other sensors, and (c) validating behaviour in ArduPilot's SITL simulator before any hardware-in-the-loop test, exactly as AERO-GCS development already does for other features.

10.4 Integration with AERO-GCS

Since AERO-GCS is our own Mission Planner–derived ground station, it is well positioned to become the operator-facing window into this system rather than treating SLAM as an invisible background process. Concretely:

- A live map panel showing the SLAM-generated occupancy grid or point cloud as it builds during flight — directly useful for a SAR operator trying to understand the interior of a structure in real time.
- A navigation-source status indicator (GPS / Vision / Fused), so the operator always knows what the EKF is currently trusting — critical for safety awareness during a GPS-to-vision handover.
- Optionally, overlaying SAR detection results (from the existing YOLOv8 pipeline) onto the SLAM-built map, so a detection made indoors can be shown at its actual position within the structure rather than as an unplaced video frame.

10.5 Phased Implementation Plan

Phase	Milestone	Activities	Status
0	Research foundation	Framework evaluation, environment setup, terminology & pipeline understanding	Complete
1	Bench validation	Static camera + companion computer rig; validate RTAB-Map mapping accuracy against known distances/objects	Next
2	Handheld / walked test	Carry the rig through an indoor space; evaluate drift, loop closure, and map quality outside a static bench	Planned
3	SITL telemetry bridge	MAVROS + EK3_SRC configuration validated against ArduPilot SITL, no physical airframe involved	Planned
4	Grounded hardware-in-the-loop	Full rig mounted on the airframe, propellers off, validating real telemetry flow end-to-end into the flight controller	Planned
5	Caged/indoor GPS-denied hover	Controlled indoor flight test (netted area) validating vision-based position hold	Planned
6	Field / SAR-relevant test	Structured test in a building- or tunnel-like environment representative of intended SAR use	Future

10.6 Use Case: SAR / GPS-Denied Operations

This work directly complements the existing SAR detection system: today, that system's detections are only as useful as the GPS/telemetry position attached to them, which is exactly what is unavailable in the environments SAR most needs — collapsed structures, tunnels, dense interiors. A working SLAM stack would let the drone build and share a local map of such a space even with no GPS at all, and would let detections be placed within that map rather than left as an unplaced video frame. That combination — local mapping plus geo-referenced detection — is a meaningfully stronger SAR product than either capability alone.

10.7 Safety & Failover Considerations

- SLAM-based position must degrade gracefully, not silently — a lost-tracking event should trigger an explicit failsafe behaviour (e.g., hold last known position and alert), not an unnoticed drift into bad state.
- Where GPS is intermittently available, GPS and vision estimates should be cross-checked rather than blindly switched between, using ArduPilot's EKF lane-switching behaviour as the underlying mechanism.
- Feature-poor environments (bare walls, smoke, low light) are the realistic failure mode for feature-based SLAM and should be treated as an expected condition to test for, not an edge case.
- Any autonomous behaviour commanded from a vision-derived position estimate should be more conservative (lower speed/acceleration limits) than the same behaviour under a trusted GPS fix, at least until the vision pipeline has a flight-proven track record on this platform.

11. Risks & Challenges

- Computational load: RTAB-Map competing with other onboard workloads (e.g., YOLOv8 SAR inference) for the same companion computer's GPU/CPU budget needs explicit profiling, not an assumption that both fit comfortably.
- Vibration & motion blur: a multirotor's vibration environment is harsher than the bench/handheld testing done so far and can meaningfully degrade feature tracking; mechanical isolation for the camera mount is likely necessary.
- Lighting and texture dependence: feature-based SLAM struggles in low light and on low-texture surfaces — a real concern for indoor SAR scenes and one worth testing explicitly rather than assuming away.
- SWaP (size, weight, power) budget: this is additive payload and compute draw on an already-loaded airframe and needs to be reconciled against current flight-time figures.
- Integration complexity: bridging a research-grade SLAM stack into a certified-feeling, safety-relevant flight-control input is a materially bigger engineering lift than getting the same stack running for visualization on a bench.

12. Future Scope

- Integrate RTAB-Map with ArduPilot via the MAVROS bridge described in Section 10.2–10.3.
- Generate real-time 3D maps usable both onboard (for obstacle avoidance) and on the ground (for operator situational awareness).
- Combine SLAM output with existing telemetry for a single, fused navigation picture rather than two parallel data streams.
- Implement autonomous obstacle avoidance using the occupancy grid / point cloud output as an input to path planning.
- Integrate the existing AI-based (YOLOv8) object/person detection pipeline with SLAM-derived positioning, so SAR detections are placed within the built map.
- Visualize localization data — pose, map, navigation-source status — natively within AERO-GCS, as detailed in Section 10.4.

13. Conclusion

This research established a solid technical foundation in SLAM concepts, Linux robotics development, ROS 2 integration, and the beginnings of autonomous navigation work relevant to our platform. It was, by design, a research and environment-preparation phase rather than a deployed system — and that sequencing was the right call, since the framework, hardware, and integration decisions made here directly shape whether the eventual onboard implementation is a straightforward bring-up or a difficult retrofit. Section 10 of this report turns that research into an actionable, phased plan for getting SLAM from a bench evaluation into a flight-tested, GPS-denied navigation capability integrated with our existing ArduPilot airframes and AERO-GCS.

14. References & Further Reading

- Campos, C. et al. — "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM" (IEEE Transactions on Robotics).
- Labbé, M. & Michaud, F. — "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation" (Journal of Field Robotics).
- ROS 2 (Jazzy Jalisco) official documentation — docs.ros.org.
- ArduPilot documentation — EKF3 external navigation / non-GPS navigation sources — ardupilot.org.
- RTAB-Map project documentation and ROS 2 integration guide — introlab.github.io/rtabmap.